

Assignment 0

In this homework assignment you can train your programming skills in Python. Make sure you implement all the algorithms in Section 2 and Section 3. They will be used heavily in the coming assignments.

Setting up your programming environment

Alternative I (Python and VS Code) You need to install an editor, for example Visual Studio Code (VS Code) and a Python interpreter. You can find a tutorial here:
<https://code.visualstudio.com/docs/python/python-tutorial>.

Alternative II (Cocalc) Create an account on CoCalc at <https://cocalc.com>. Create a new project. Inside of the project you can to a new Jupyter-file select a kernel, recommended is *Python 3(system wide)*.

1 First steps in Python

- (a) We begin assigning a value to a variable. You can also use a variable to reassign itself.

```
x=2
print(x)
y=x+2
print(y)
y=y*y
print(y)
```

- (b) In this course we will use the % symbol a lot, $a\%b$ gives the remainder of a when divided by b . The symbol // is used for integer division, $a//b$ gives the quotient of a when divided by b .

```
a=128
b=62
print(a/b)
print(a//b)
print(a%b)
```

- (c) We can also use so called casting to force an output type we want

```
a=128
b=62
print(float(a/b))
print("The quotient of", a,"divided by", b, "is", int(a/b))
```

2. Let us now look at lists (or arrays).

- (a) We start initializing some lists

```
list1=[] #Empty list
list2=[1,2,3,4]
print(list1)
print(list2)
```

- (b) We can easily get an element from the list by using brackets

```
print(list2[0])
print(list2[2])
print(list2[-1])
print(list2[-2])
```

What happens if we try `list2[4]` or `list1[0]`? We can also change an element of a list easily

```
list2[2]=7
print(list2)
```

- (c) You can concatenate two lists (or add elements to the beginning or the end of a list) by using `+`. It is also possible to use the `append` method to add elements in the end of a list. We can use the `sort()` method to sort a list.

```
list3=[5,6]
print(list2+list3)
print([0]+list2+[5])
list2.append(5)
print(print(list2))
list2.sort()
print(list2)
```

Try also the `sort()` method:

```
list2.sort()
print(list2)
```

3. In this exercise we look at the `if` statements. Note the colon and the indentation in the code!

- (a) `x=3`
- ```
if x%2==0:
 print(x, "is an even number")
else:
 print(x, "is an odd number")
```

- (b) We can also use logical operators like `and` and `or`

```
a=3
b=-2
if a>0 and b>0:
 print("Both", a, "and" , b, "are positive")
else:
 print("At least one of", a, "and", b, "is not positive")

if a>0 or b>0:
 print("At least one of", a, "and", b, "is positive")
else:
 print("Neither", a, "nor" , b, "are positive")
```

- (c) We can also check if an element is in a list or not.

```
A=[1,2,3,4,5]
if 1 in A:
 print("1 is in the list")
```

4. There are two ways of constructing loops in Python, **for** and **while**

- (a) A **for** loop runs through the values of a list

```
for i in [1,3,5,7,9]:
 print(i)
for i in range(0,5):
 print(i)
```

- (b) Let's create a list of integer points

```
listOfPoints=[]
for i in range(0,3):
 for j in range(0,3):
 listOfPoints.append([i,j])
print(listOfPoints)
```

- (c) The **while** loops works similar to the **if** statements. It will run a piece of code until a statement is false. The following code will produce a list of non-negative even integers less than 20.

```
x=0
evenList=[]
while x<20:
 evenList.append(x)
 x=x+2
print(evenList)
```

- (d) We can (by mistake) end up in an infinite loop. If this happens we have to stop the evaluation of the program.

```
x=1
while x>0: #infinite loop
 print(x)
 x=x+1
```

- (e) What will the following code produce?

```
a=1
m=7
upperLimit=100
listOfNumbers=[]
x=a
while x<upperLimit:
 if x%m==a:
 listOfNumbers.append(x)
 x=x+1
print(listOfNumbers)
```

5. In Python we can define our own functions. This we will use a lot in the homework assignments in this course.

- (a) We can define a function without input

```
def quoteDiffieHellman():
 print("We stand today on the brink of a revolution in cryptography.")
```

You can use the function by simple writing

```
quoteDiffieHellman()
```

and then run it.

- (b) ... and we can define functions with inputs

```
def divisor(a,b): #is a a divisor of b?
 if b%a==0:
 return True
 else:
 return False
```

Run `divisor(a,b)` for some different values of  $a$  and  $b$ .

- (c) We can also use lists as inputs

```
def findDivisors(listOfNumbers,m) #find divisors of m in listOfNumbers
 listOfDivisors=[]
 for i in listOfNumbers:
 if divisor(i,m):
 listOfDivisors.append(i)
 return listOfDivisors
```

Try this function out by running the following code

```
m=271627632
findDivisors(range(1,100),m)
```

## 2 Algorithms from Number Theory

Implement the following basic algorithms in Python and try them out.

6. The fast powering algorithm

```
1 #Compute the power of a^n modulo m
2 def fast2Power(a,n,m):
3 res=1
4 while n>0:
5 if n%2==1: #If the bit is 1 multiply by the corresponding square
6 res=(res*a)%m
7 a=(a*a)%m
8 n=n//2
9 return res
```

## 7. Compute

- (a)  $34^{37} \bmod 47$ ,
- (b)  $61462516^{2^{14}7246} \bmod 561623171$ .

## 8. The extended euclidean algorithm

```

1 #Output [r,s,t] satisfying s*a+t*b=r=gcd(a,b)
2 def extendedGCD(a,b):
3 r0,r=a,b
4 s0,s=1,0
5 t0,t=0,1
6 while(r>0):
7 tempr,temps,tempt=r,s,t
8 q=r0//r
9 r,s,t=r0-q*r,s0-q*s,t0-q*t
10 r0,s0,t0=tempr,temps,tempt
11 return [r0,s0,t0]
```

## 9. Compute

- (a)  $\gcd(543, 705)$
- (b)  $\gcd(173676, 818645)$
- (c)  $\gcd(98748791589859238764602, 785168804371787933116521)$
- (d)  $\gcd(3107289384021020334845112174217281, 4356355227969236398381420990623887)$

## 10. Multiplicative inverse

```

1 #Output the multiplicative inverse of a modulo p
2 def multInverse(a,p):
3 result=extendedGCD(a,p)
4 if result[0]!=1: #Error message if a and p are not relatively prime
5 s="Numbers needs to be relatively prime"
6 return s
7 inv=result[1]%p
8 return inv
```

## 11. Find the multiplicative inverses. Check also that your results are valid.

- (a)  $7^{-1} \bmod 53$
- (b)  $1066^{-1} \bmod 2027$
- (c)  $72161726471264212144^{-1} \bmod 217264871624871628472652659$

12. Expansion of a number  $n$  i base  $m$ . Make sure that you understand in which order the digits come in the list.

```

1 #Finds the m-ary expansion of n
2 def getExpansion(n,m):
3 listOfDigits=[]
4 while n>=m:
5 digit=n%m
6 listOfDigits.append(digit)
7 n=(n-digit)//m
8 listOfDigits.append(n)
9 return listOfDigits
```

13. (a) Find the binary expansion of 685.  
 (b) Find the expansion of 1456 in base 5.  
 (c) Find the expansion of 123456789 in base 16.
14. Chinese remainder theorem

```

1 #Chinese remainder theorem
2 def CRT(remainderList,modulusList):
3 M=1
4 l=0
5 for m in modulusList: #Computes the product of the moduli
6 M=M*m
7 l=l+1
8 if l!= len(remainderList):
9 print("Length of lists must be the same")
10 return None
11 x=0
12 for i in range(0,l):
13 m=modulusList[i]
14 Mi=M//m
15 yi=multInverse(Mi,m)
16 x=(x+remainderList[i]*Mi*yi)%M
17 return x

```

15. Test the algorithm by solving the classical problem

$$\begin{cases} x \equiv 2 \pmod{3}, \\ x \equiv 3 \pmod{5}, \\ x \equiv 2 \pmod{7}. \end{cases}$$

### 3 Algorithms for converting between strings and integers

We will use the following algorithms when we convert text to integers and vice versa.

16. Convert a string to an integer

```

1 def textToInt(s):
2 n=0
3 k=0
4 for i in s:
5 n=n+ord(i)*(256**k)
6 k=k+1
7 return n
8

```

17. Convert an integer to a string

```

1 def intToText(n):
2 t=getExpansion(n,256)
3 myString=''
4 for i in t:
5 myString=myString+chr(i)
6 return myString
7

```

18. Try these algorithms out by converting a string, for example

`We stand today on the brink of a revolution in cryptography!`

to an integer. Then convert it back to a string.